22

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

요청 파이프라인의 실행 순서

제2부 NestJS 핵심 · 6장 요청 파이프라인

요청 하나가 핸들러 앞뒤로 지나는 관문과 그 순서



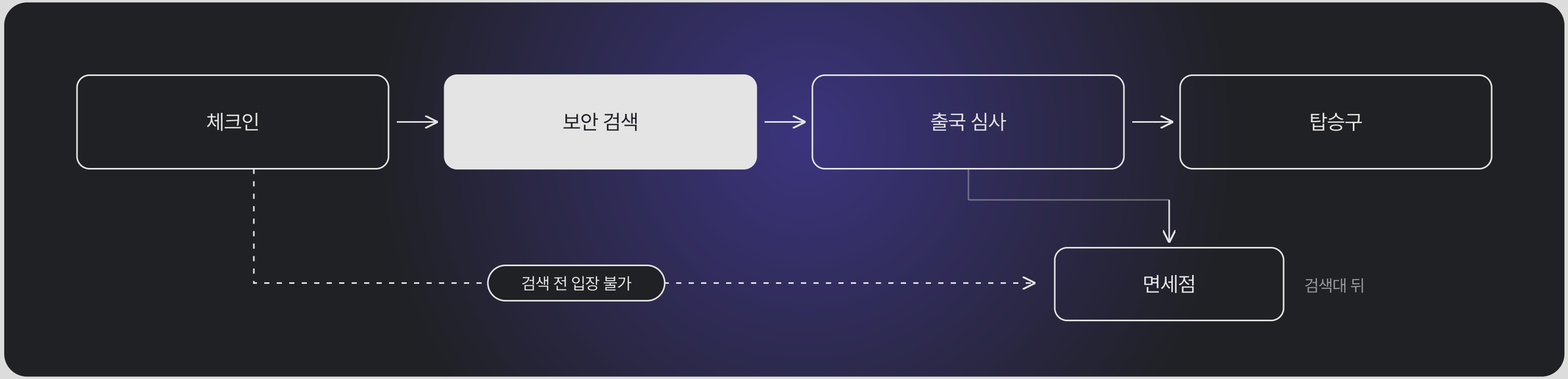
© 한상훈 NestJS 백엔드 마스터

공항 출국 순서

→ 체크인 → 보안 검색 → 출국 심사 → 탑승구

→ 보안 검색 전에 면세점 입장 불가

→ 순서는 공항이 정함, 승객이 고르지 않음



© 한상훈 NestJS 백엔드 마스터

면세점 직원이 보는 고객

면세점 직원

보안 검색과 출국 심사를 통과한 고객만 봄

검색대에서 돌아간 고객

면세점 기록에 없음

관문

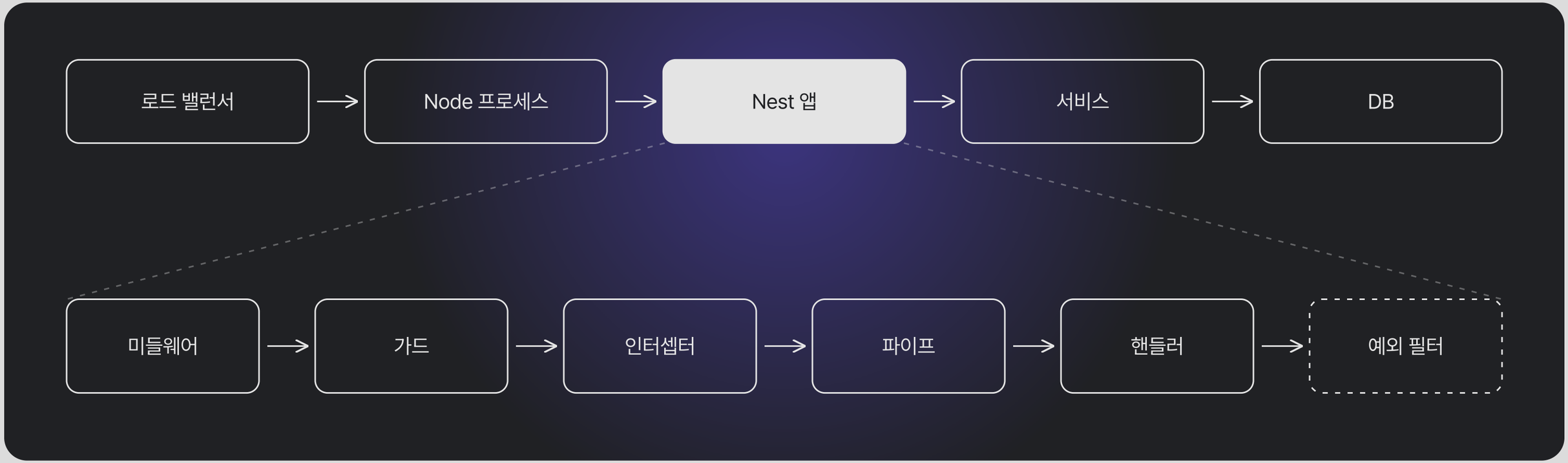
관문마다 보는 고객이 다름

© 한상훈 NestJS 백엔드 마스터

요청 일생 그림 속 관문

→ 로드 밸런서 → Node 프로세스 → Nest 앱 안의 관문 → 서비스 → DB

→ Nest 앱 안: 미들웨어, 가드, 인터셉터, 파이프, 핸들러, 예외 필터



© 한상훈 NestJS 백엔드 마스터

관문 다섯 개

- → 미들웨어 — 모든 요청이 맨 먼저 지나는 함수
- → 가드 — 이 요청을 들여보낼지 결정
- → 인터셉터 — 핸들러 앞뒤를 감싸는 코드
- → 파이프 — 핸들러 인자 하나를 바꾸거나 검사
- → 예외 필터 — 던져진 예외를 HTTP 응답으로 변환

© 한상훈 NestJS 백엔드 마스터

전부 Nest가 만드는 클래스

→ 각 관문: 정해진 메서드 하나를 가진 클래스

→ 가드 `canActivate`, 인터셉터 `intercept`, 파이프 `transform`, 예외 필터 `catch`

→ Nest가 인스턴스를 만들고 생성자에 의존성 주입

→ 미들웨어: 핸들러가 정해지기 전, req·res·next만 받음

→ 가드·인터셉터·파이프·예외 필터: 핸들러가 정해진 뒤, 어느 핸들러인지 앎

"객체를 쓰는 쪽은 받기만 하고, 만드는 일은 Nest가 맡습니다."

© 한상훈 NestJS 백엔드 마스터

문서 목차와 실행 순서

순서	공식 문서 목차	실제 실행
1	미들웨어	미들웨어
2	예외 필터	가드
3	파이프	인터셉터 (앞)
4	가드	파이프
5	인터셉터	핸들러 → 인터셉터 (뒤)
6		예외가 나면 예외 필터

데이터 블록: docs.nestjs.com Overview 목차 기준 (2026-10-26), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

관문마다 번호를 찍는 trace 모듈

```ts
@Injectable()
export class TraceMiddleware implements NestMiddleware {
  use(req: any, res: any, next: () => void) {
    log(`1 미들웨어 ${req.method} ${req.originalUrl}`);
    next();
  }
}
```

```ts
@Injectable()
export class TraceInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler) {
    log('3 인터셉터 (앞)');
    return next.handle().pipe(tap(() => log('6 인터셉터 (뒤)')));
  }
}
```

```ts
@Injectable()
export class TraceGuard implements CanActivate {
  canActivate(context: ExecutionContext) {
    const req = context.switchToHttp().getRequest();
    log(`2 가드 (params.id: ${typeof req.params.id} ${JSON.stringify(req.params.id)})`);
    return req.headers['x-block'] !== 'guard';
  }
}
```

trace/trace.ts 일부, 파이프·필터·메서드 인터셉터는 부록

© 한상훈 NestJS 백엔드 마스터

TraceController

trace/trace.controller.ts

```ts
@Controller('trace')
@UseGuards(TraceGuard)
@UseInterceptors(TraceInterceptor)
@UseFilters(TraceFilter)
export class TraceController {
  @Get(':id')
  @UseInterceptors(MethodInterceptor)
  find(@Param('id', TracePipe, ParseIntPipe) id: number) {
    log(`5 핸들러 id=${id}`);
    if (id === 0) throw new Error('핸들러에서 터짐');
    return { id };
  }
}
```

→ 미들웨어: TraceModule의 `configure`에서 모든 경로에 적용

→ `x-block: guard` 헤더 → 가드가 `false`

© 한상훈 NestJS 백엔드 마스터

GET /trace/7의 로그

curl

```
$ curl localhost:4022/trace/7
{"id":7}
```

서버 로그

```
1 미들웨어 GET /trace/7
2 가드 (params.id: string "7")
3 인터셉터 (앞)
3b 메서드 인터셉터 (앞)
4 파이프 (param id=7)
5 핸들러 id=7
6a 메서드 인터셉터 (뒤)
6 인터셉터 (뒤)
```

© 한상훈 NestJS 백엔드 마스터

파이프는 인자마다, 적은 순서대로

```
@Param('id', TracePipe, ParseIntPipe)
```

TracePipe → ParseIntPipe

→ TracePipe가 본 값: 문자열 "7"

→ 핸들러가 받은 값: 숫자 7

→ 인자가 여럿이면 인자마다 각자의 파이프 (예: 회차 번호, 좌석 번호, 본문)

→ 핸들러 실행 = 모든 인자가 파이프 통과

→ ParseIntPipe를 앞에 적으면 TracePipe가 숫자 7을 받음

© 한상훈 NestJS 백엔드 마스터

가드가 거절한 요청

curl

```
$ curl -H 'x-block: guard' localhost:4022/trace/7
{"type":"about:blank","title":"Forbidden","status":403,
 "detail":"Forbidden resource","instance":"/trace/7"}
```

서버 로그

```
1 미들웨어 GET /trace/7
2 가드 (params.id: string "7")
7 예외 필터 (ForbiddenException)
```

→ 가드가 `false` → Nest가 ForbiddenException을 던짐

© 한상훈 NestJS 백엔드 마스터

파이프에서 실패한 요청

curl

```
$ curl localhost:4022/trace/abc
{"type":"about:blank","title":"Bad Request","status":400,
 "detail":"Validation failed (numeric string is expected)",
 "instance":"/trace/abc"}
```

서버 로그

```
1 미들웨어 GET /trace/abc
2 가드 (params.id: string "abc")
3 인터셉터 (앞)
3b 메서드 인터셉터 (앞)
4 파이프 (param id=abc)
7 예외 필터 (BadRequestException)
```

© 한상훈 NestJS 백엔드 마스터

핸들러가 던진 요청

curl

```
$ curl localhost:4022/trace/0
{"type":"about:blank","title":"Internal Server Error","status":500,
 "detail":"요청을 처리하지 못했습니다.","instance":"/trace/0"}
```

서버 로그

```
1 미들웨어 GET /trace/0
2 가드 (params.id: string "0")
3 인터셉터 (앞)
3b 메서드 인터셉터 (앞)
4 파이프 (param id=0)
5 핸들러 id=0
7 예외 필터 (Error)
ERROR [ProblemDetails] Error: 핸들러에서 터짐
```

© 한상훈 NestJS 백엔드 마스터

없는 주소

curl

```
$ curl localhost:4022/nope
{"type":"about:blank","title":"Not Found","status":404,
 "detail":"Cannot GET /nope","instance":"/nope"}
```

서버 로그

```
1 미들웨어 GET /nope
```

→ 라우트를 못 찾음: 가드·인터셉터·파이프·핸들러 없음

→ 404 응답: 전역 `ProblemDetailsFilter`

© 한상훈 NestJS 백엔드 마스터

어디서 끊겼나

요청	마지막으로 찍힌 관문	응답
`x-block: guard`	2 가드	403
`/trace/abc`	4 파이프	400
`/trace/0`	5 핸들러	500
`/nope`	1 미들웨어	404

© 한상훈 NestJS 백엔드 마스터

들어갈 때와 나올 때

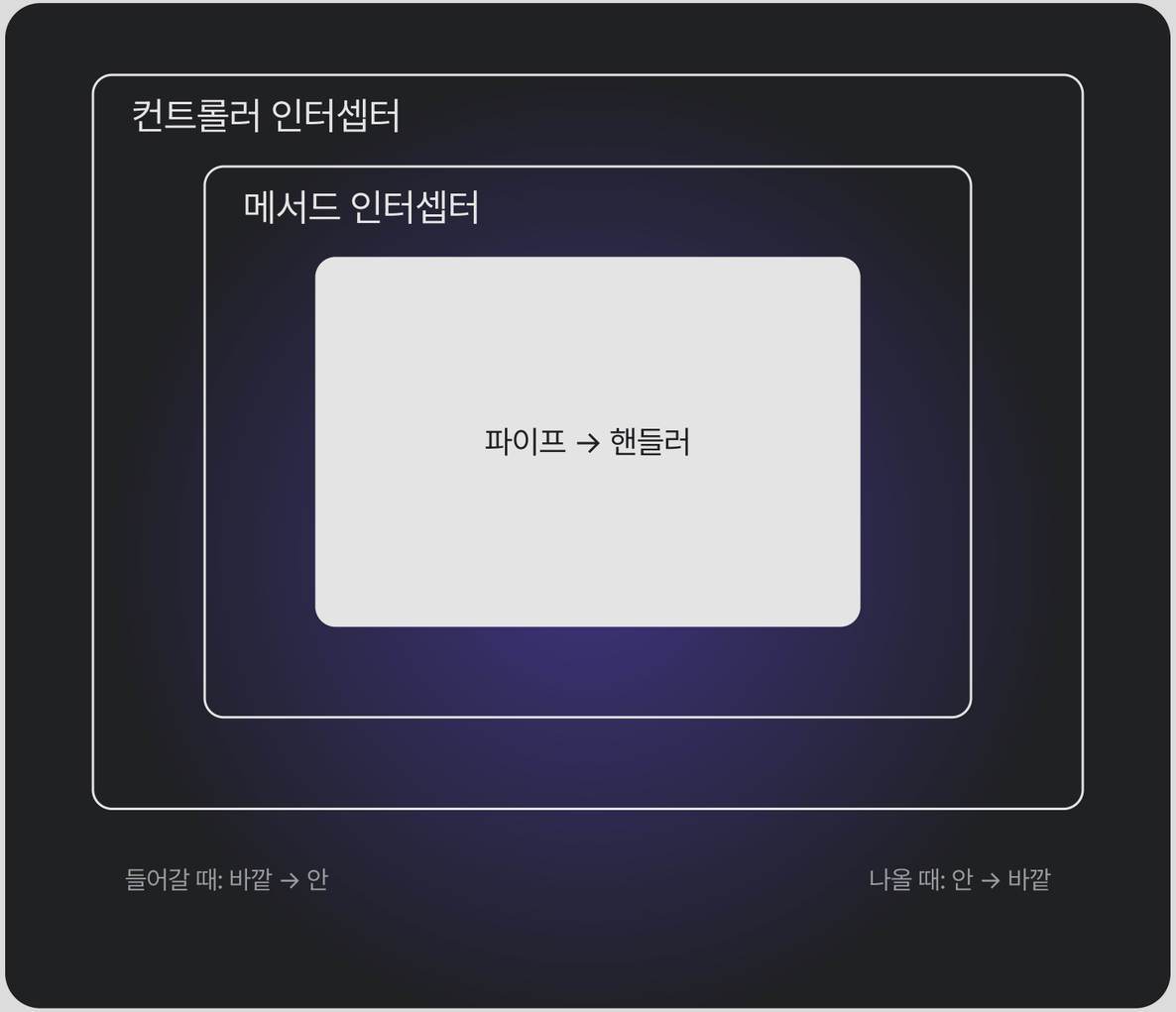
→ 들어갈 때: 컨트롤러 인터셉터 → 메서드 인터셉터 → 파이프 → 핸들러

→ 나올 때: 메서드 인터셉터 → 컨트롤러 인터셉터

→ 바깥에 건 것이 먼저 시작해서 나중에 끝남

→ 가정: 바깥에서 잰 시간 12ms, 안쪽에서 잰 시간 10ms → 차이 2ms는 안쪽 인터셉터 자신의 시간

→ 처리 시간 측정: 가장 바깥(전역)에



© 한상훈 NestJS 백엔드 마스터

예외가 나면 인터셉터 뒤쪽은 없음

→ 정상	6a 메서드 인터셉터 (뒤), 6 인터셉터 (뒤)
→ 400·500	뒤쪽 줄 없이 7 예외 필터
→ 인터셉터의 뒤쪽 코드	핸들러가 값을 돌려줄 때만 실행
→ 예외를 응답으로 바꾸는 곳	예외 필터

© 한상훈 NestJS 백엔드 마스터

이 장의 강 순서

실행 순서	이 장에서 다루는 순서
미들웨어	미들웨어
가드	가드와 실행 컨텍스트
인터셉터 (앞)	파이프
파이프	인터셉터
핸들러, 인터셉터 (뒤)	예외 필터
예외 필터	커스텀 데코레이터와 메타데이터

인터셉터: 파이프와 핸들러를 감싸는 쪽이라 감싸이는 파이프 다음에

© 한상훈 NestJS 백엔드 마스터

처리 시간 로그를 인터셉터에 두면

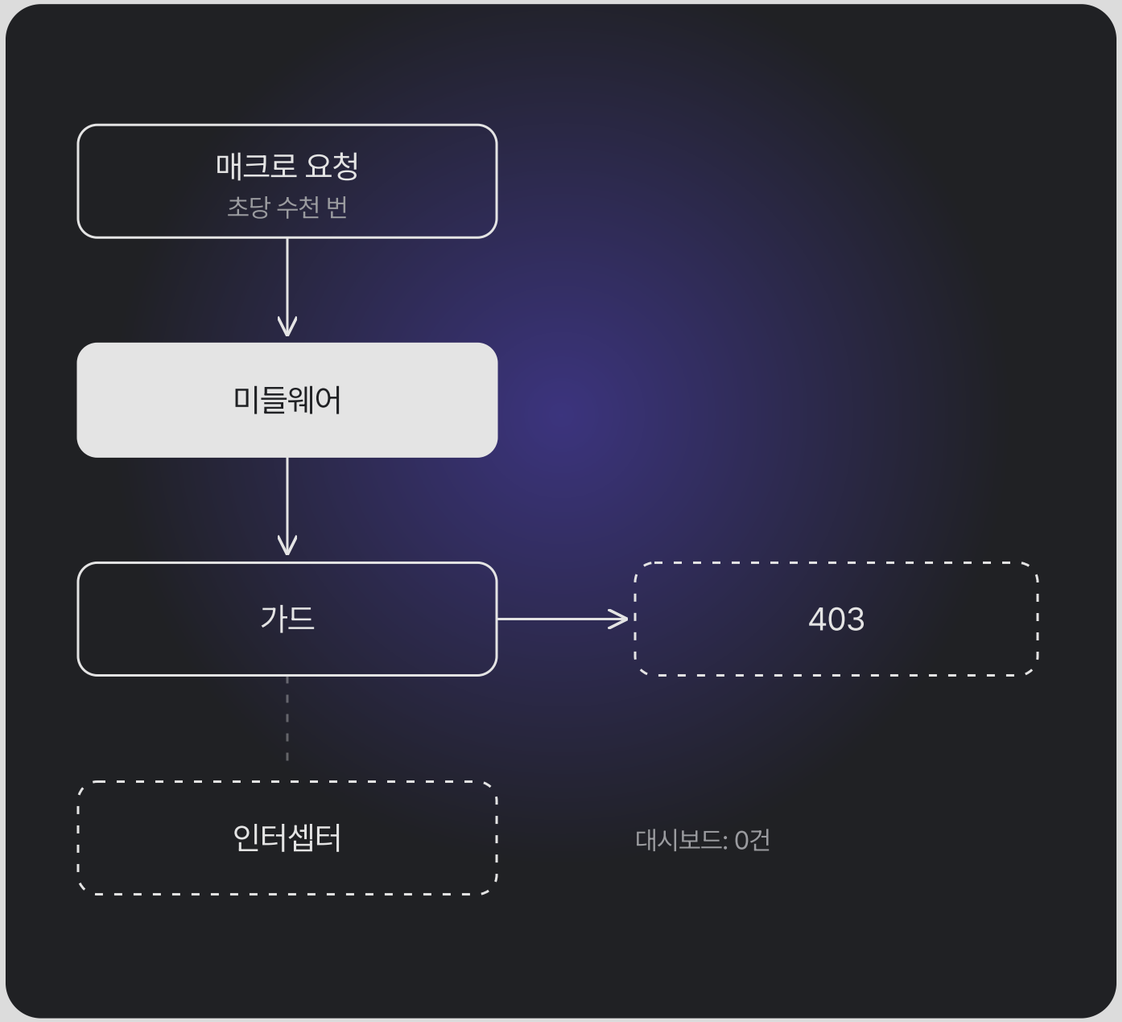
→ 가정: 응답 시간 로그를 인터셉터에서 기록

→ 오픈 직전 매크로가 403을 초당 수천 번 받음

→ 처리 시간 대시보드: 403 요청 0건

→ 이유: 인터셉터는 가드를 통과한 요청만 봄

→ 모든 요청이 필요한 기록: 미들웨어



© 한상훈 NestJS 백엔드 마스터

가드가 본 params.id

서버 로그

```
2 가드 (params.id: string "7")
4 파이프 (param id=7)
5 핸들러 id=7
```

→ 가드 시점: 파이프 전, 경로 값·쿼리·본문 모두 받은 그대로

→ 가드에서 `req.params.id === 7` → 늘 false

→ 가드에서 본문 필드 → 검증 전 값

© 한상훈 NestJS 백엔드 마스터

성공만 감싸는 응답

응답	인터셉터로 { data } 감싸기
200 성공	`{ "data": { "id": 7 } }`
400·403·500	감싸지지 않음, Problem Details 그대로

클라이언트가 받는 본문 모양: 두 가지

© 한상훈 NestJS 백엔드 마스터

인터셉터 앞에서 본 본문

→ 인터셉터 앞부분: 파이프보다 먼저 실행

→ 이 시점의 `req.body`: 검증·변환 전, 클라이언트가 보낸 그대로

→ 가정: 감사 로그 인터셉터가 본문 전체를 기록

→ 결과: 거절될 요청의 엉터리 값, 받지 않을 필드까지 로그에 남음

→ 기록할 값: 핸들러가 받은 인자 또는 핸들러 뒤의 결과

© 한상훈 NestJS 백엔드 마스터

관문을 고르는 질문 두 개

질문	관문
이 일은 어떤 요청까지 봐야 하나	모든 요청이면 미들웨어
어느 핸들러로 가는지 알아야 하나	알아야 하면 가드·인터셉터
핸들러 인자의 값이 바뀌어야 하나	파이프
실패를 응답으로 바꾸나	예외 필터

© 한상훈 NestJS 백엔드 마스터

전역, 컨트롤러, 메서드

→ 거는 곳 세 군데: 서버 전체, 컨트롤러 클래스, 핸들러 메서드

→ 가드·인터셉터·파이프: 전역 → 컨트롤러 → 메서드 순서로 실행

→ 파이프는 그 뒤에 인자별 파이프

→ 예: 전역 점검 중 가드 → 컨트롤러 직원 키 가드 → 메서드 실장 가드

→ 바깥 가드가 막으면 안쪽 가드는 실행 안 됨



© 한상훈 NestJS 백엔드 마스터

전역으로 거는 두 방법

방법	코드	생성자 주입
main.ts	`app.useGlobalGuards(new XGuard())`	없음 (직접 new)
모듈 프로바이더	`{ provide: APP_GUARD, useClass: XGuard }`	있음 (Nest가 생성)

→ 같은 방식: `useGlobalInterceptors`·`APP_INTERCEPTOR`, `useGlobalPipes`·`APP_PIPE`, `useGlobalFilters`·`APP_FILTER`

→ `APP_GUARD`: 어느 모듈에 적어도 서버 전체에 적용

© 한상훈 NestJS 백엔드 마스터

요청 파이프라인

"요청은 미들웨어, 가드, 인터셉터, 파이프를 지나 핸들러에 닿고,
어디서 던진 예외든 예외 필터가 받습니다."